Concord

The app that unifies clinical trial systems source data





Sean Pun
Aleksander Kasperski
Timothy Wu
Dennis Chen

The challenge

Clinical trials generate data across separate software systems:

- Inconsistent master data
- Inflexible & slow reporting
- Regulatory, compliance & access barriers

To extract insights, analysts spend WEEKS combining the sources and formatting them to a standard, which can still be prone to errors.

Our goal

To be able to ingest the unharmonized raw data from 5 clinical trial systems and output a user interface for analysts/engineers to query easily, adhering to FAIR principles

The same site has five names:

CTMS trial management - SITE-0708

EDC trial database - 708

IRT drug supply - US708

Safety - Seattle Center for Clinical Trials (sic)

Finance - SEATTLE CENTER FOR CLINICAL TRIALS, LLC

Introducing Concord:
the app to automate harmonization

Harmonizer

Ingests the data and standardizes to be joined into one query-able table

implements data rules to catch problems

Server

home of the REST API for query endpoints

Translates user queries into SQL with Gemini

Implements different methods of user querying

Client

User Interface connects with server logic to allow users to query data and see lineage, crosswalk, quality, and access

Harmonization (Medallion Schema)

1

Ingestion of Data

Ingest the RAW data into DuckDB from 5 systems: CTMS, EDC, IRT, Financial, Safety

BRONZE

2

Transform into Schema

Transform the data so that IDs have a common format with regex.

Ex. ID is standardized to 4 digit

SILVER

3

Harmonize the IDs

Across all silver tables, JOIN by the master id based off the CTMS source data

MASTER

4

Clean

Remove duplicates and format properly. Runs quality checks. Final cleanup that the program can query.

GOLD

Standardization

Problems with transformation and the solutions

Problem:
Safety and Finance Systems do not have a numeric site ID

Solution:

Implement Jaccard set scoring by comparing site names and assigning a score based on how similar the site names were from the master site (CTMS)

Problem:
Incomplete dates (e.g. 2000-02)

Solution:
Implement a flagging system to warn against incomplete data

Server/Backend

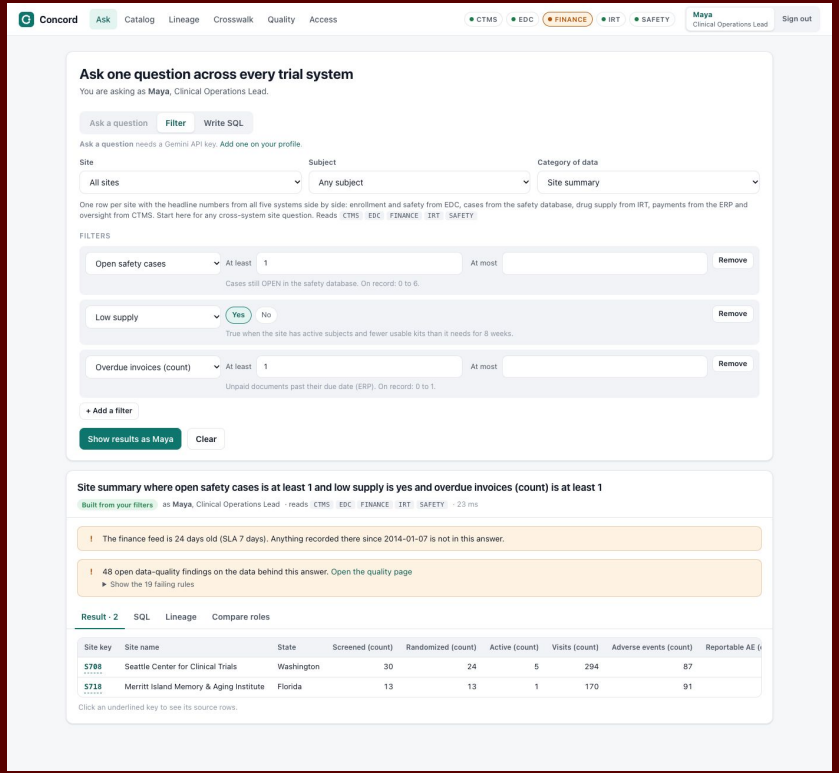
Technologies: fastAPI, DuckDB, Python

Contains:

Connection to the Database

REST API (fastAPI) for query endpoints

General Logic for querying the database for lineage, crosswalk, and quality

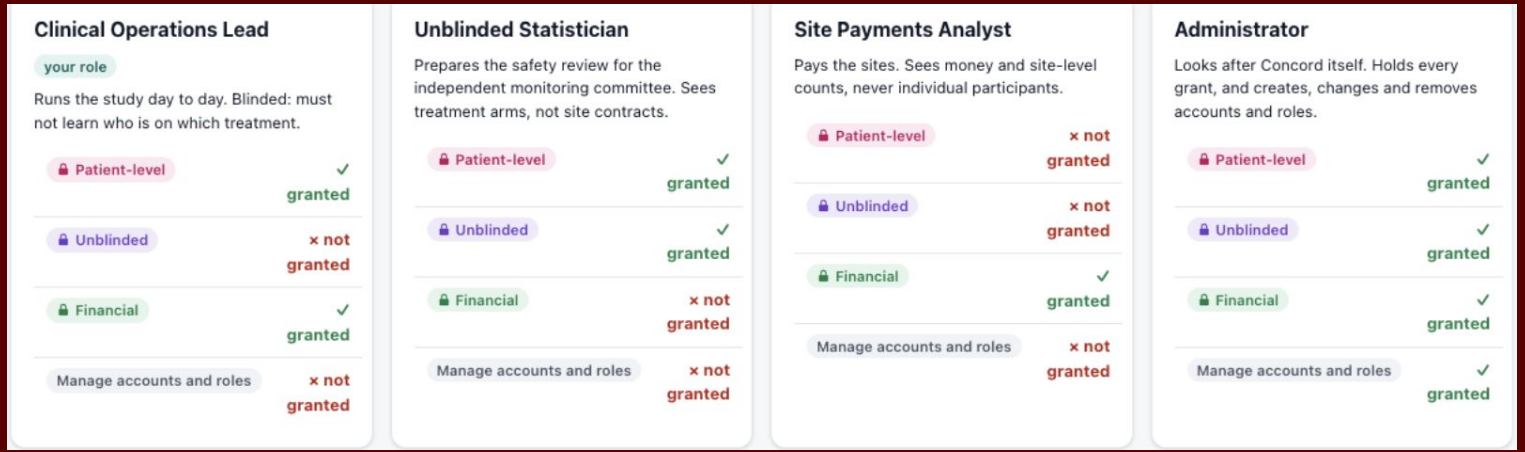


Continued

Login Authentication for RBAC

Query logic based on RBAC configurations

(Filtering, Gemini, SQL direct query)



Client

Technologies:

- Next.js
- Tailwind CSS
- TypeScript

- Ask queries in natural language
- Write SQL for faster speed
- View all messy or clean data
- See how the data was harmonized
- Role-based restricted access

Future Ponderence

File System & Medallion Access

- File browser for raw feeds and Medallion layers (Bronze/Silver/Master/Gold)
- Raw CSV/JSON and structured Parquet toggles

Zero-Downtime Data Lifecycle

- Dynamic ingestion and archiving without server restarts
- Live Gold table updates via Delta Lake transactions

Big Data & PySpark Scaling

- Offload heavy crosswalk transformations to PySpark
- Distributed processing for out-of-core, multi-terabyte datasets





Thank

You